CSD2183 Data Structures: Project 2

Area- and Topology-Preserving Polygon Simplification

Michael T. Gastner

AY2025/26, Trimester 2

In this project, you will design and implement an algorithm to simplify the boundary of a planar polygon with one outer boundary ("exterior ring") and an arbitrary number of holes ("interior rings"). Figure 1 shows an example of such a polygon simplification.

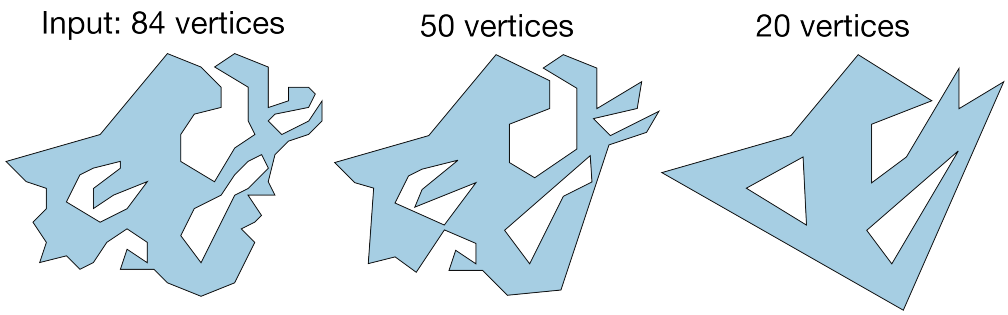

Figure 1: Example of area- and topology-preserving polygon simplification, implemented with the method described in Kronenfeld et al. (2020).

1 Objective

Polygons are a common representation of geographic data, but complex boundaries with many vertices can be computationally expensive to process and visualize. The objective of this project is to replace an arbitrary input polygon, given by an exterior ring and zero or more interior rings, with a simplified polygon that uses fewer vertices while preserving area constraints exactly. Specifically, you must preserve the area enclosed by the exterior ring and by each interior ring (hole), individually; equivalently, preserve the polygon's filled area and each hole's area. The polygon's topology must remain unchanged: the number of rings must be the same, and no ring may intersect itself or any other ring. Among all simplified polygons that satisfy these constraints, the algorithm should minimize areal displacement, defined as the area of the symmetric difference between the original and simplified polygon. Subject to these geometric and topological constraints, you should design your data structures and algorithms so that your implementation remains efficient even for polygons with a large number of vertices.

Kronenfeld et al. (2020) describe an Area-Preserving Segment Collapse (APSC) algorithm that addresses this problem. The paper is available on xSITe. You should implement APSC as the core of your solution and explore enhancements that improve performance or accuracy.

Data structures focus. Your solution should demonstrate thoughtful choices of representations and update mechanisms, such as:

- a suitable ring representation (e.g., a doubly connected edge list),
- a priority queue (or equivalent structure) for selecting candidate collapses,
- a spatial index (or equivalent acceleration structure) for intersection checks,
- local/incremental updates after each collapse (rather than global recomputation), and
- asymptotic reasoning supported by profiling measurements.

2 Applications

Modern mapping platforms, geographic information system (GIS) databases, and game worlds often contain thousands of polygonal regions that must be drawn, queried, and updated in real time. Simplifying these polygons while preserving area and topological relationships enables web maps and GIS applications to render and analyze large administrative or cadastral datasets more quickly, without changing associated geospatial attributes. In simulations and games, area- and topology-preserving simplification of territories and navigation regions can reduce rendering, collision detection, and pathfinding costs while keeping application-relevant areas and connectivity unchanged.

3 Definitions

Polyline: A polyline (also called a polygonal chain) is a connected sequence of straight edges (line segments) defined by an ordered list of vertices (points). A polyline is not necessarily closed; that is, the first and last vertices need not coincide.

Polygon: A polygon is a planar region whose boundary consists of one or more closed polylines (called rings). In a vertex-list representation, a ring is typically treated as closed by connecting the last vertex back to the first (i.e., the first vertex need not be repeated at the end of the list).

Simple polygon:
A polygon is called simple if its boundary does not intersect itself and contains no holes. Note that this notion of simplicity is distinct from "polygon simplification," which refers to the process of reducing the number of vertices in a polygonal shape.

Polygon with holes:
A polygon with holes consists of one outer boundary (exterior ring) and zero or more holes (interior rings), each of which must be a simple polygon. All interior rings must be contained entirely within the exterior ring and must not intersect any other ring. Figure 1 shows an example of a polygon with two holes.

Areal displacement:
When a polyline between two points is replaced with a different polyline that has the same endpoints, the areal displacement is the (unsigned) area enclosed between the old and new polylines. Figure 2 illustrates the concept: the polyline $A \to B \to C \to D$ is replaced with $A \to E \to D$, and the areal displacement is the sum of the areas to the left and right of the original path.

Topology:

In the context of this project, the topology of a polygon with holes refers to the combinatorial structure of its rings. A simplification preserves topology if the simplified polygon has the same number of rings, each ring remains simple (i.e., non-self-intersecting), and no ring intersects any other ring.

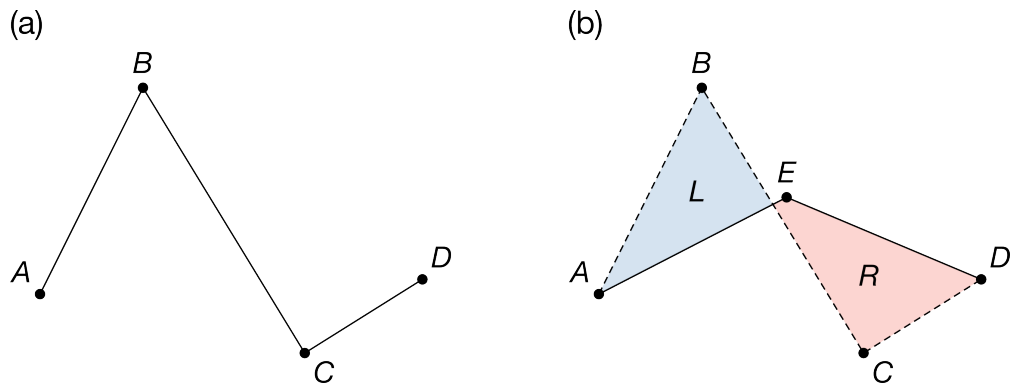

Figure 2: Illustration of areal displacement. (a) Original polyline $A \to B \to C \to D$. (b) The polyline is simplified to $A \to E \to D$ (solid), where E does not need to lie on the original polyline. The areal displacement is the combined area of the left (L, blue) and right (R, red) regions between the original polyline (dashed) and the simplified polyline.

4 Problem Specification

4.1 Input

Target number of vertices n:

An integer specifying the desired maximum number of vertices in the simplified polygon (the total across all rings). If the input polygon already has n or fewer vertices, no simplification is needed. The algorithm should reduce the vertex count to at most n while preserving area to within floating-point tolerance and maintaining topological validity. If no valid simplification with n or fewer vertices exists (because further vertex removal would inevitably violate a topological or area constraint), the algorithm should reduce the vertex count as far as possible while still satisfying all constraints.

Input polygon:

Use the following comma-separated values (CSV) format, where the exterior ring (with `ring_id` 0) is oriented counterclockwise and each interior ring is oriented clockwise. In your code, you may assume that the input polygon is topologically valid (i.e., all rings are simple and do not intersect each other, and all interior rings are contained within the exterior ring).

```
ring_id,vertex_id,x,y
0,0,-0.5,-1.0
0,1,1.5,-1.0
0,2,1.5,1.0
0,3,-0.5,1.0
1,0,-0.2,0.5
1,1,0.5,0.5
1,2,0.5,-0.5
2,0,0.6,-0.2
2,1,1.0,0.2
2,2,1.2,0.2
2,3,1.4,-0.2
2,4,1.2,-0.8
```

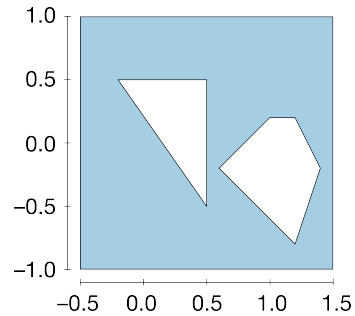

Figure 3: Example polygon with two holes, corresponding to the CSV data on the left. The exterior ring has `ring_id` 0, and the two interior rings (holes) have `ring_id` 1 and 2, respectively.

4.2 Implementation

- Use C++ (standard C++17 or later) compiled with `g++` on a Unix-like system (macOS or Linux; Windows users may use WSL).
- Include a `Makefile` at the root of your repository so that running `make` with no arguments builds an executable called `simplify` in the repository root.
- The executable must accept the following command-line arguments:

  ```
  ./simplify <input_file.csv> <target_vertices>
  ```

  where `<input_file.csv>` is the path to the input CSV file and `<target_vertices>` is the desired maximum number of vertices n.
- Aim for asymptotically efficient designs and avoid unnecessary recomputation. Assessment will emphasize algorithmic and data-structure choices rather than low-level micro-optimizations. Performance measurements and comparisons must be conducted in single-threaded mode.
- You may use data structures from third-party C++ libraries (e.g., CGAL for computational geometry) or provide your own data structures. However, you must clearly document any dependencies and ensure that your code can be built and run by others without requiring proprietary software or licenses. If you use third-party libraries, include instructions for installing them in a `README.md` file.

4.3 Output

The program must print its output to the console (standard output) in **exactly** the format shown below. No other output should be printed to standard output. Any debug, logging, or status messages must be printed to standard error.

- A header line: `ring_id,vertex_id,x,y`
- One line per vertex of the simplified polygon, in the same CSV format as the input. The ring with `ring_id` r in the output must be the simplified version of the ring with `ring_id` r in the input. Area must be preserved exactly (within floating-point tolerance), and topology must be maintained.

- `ring_id` values in the output must start at 0, be contiguous (0, 1, 2, ...), and rows must be grouped by `ring_id` in ascending order.
- Within each ring, `vertex_id` must start at 0 and increase contiguously by 1 with no gaps.
- The first vertex of a ring must not be repeated at the end of that ring in the CSV output.
- Three final lines reporting areas and areal displacement:

  ```
  Total signed area in input:  <value>
  Total signed area in output:  <value>
  Total areal displacement:  <value>
  ```

  where each `<value>` is in scientific notation. The total signed area is the algebraic sum of the signed areas of all rings (positive for counterclockwise exterior rings, negative for clockwise interior rings). The input and output values must match to within floating-point tolerance.

Example output:

```
ring_id,vertex_id,x,y
0,0,-0.5,-1
0,1,1.5,-1
0,2,1.5,1
0,3,-0.5,1
1,0,-0.2,0.5
1,1,0.5,0.5
1,2,0.5,-0.5
2,0,0.6,-0.2
2,1,1.1,0.3
2,2,1.4,-0.2
2,3,1.2,-0.8
Total signed area in input: 3.210000e+00
Total signed area in output: 3.210000e+00
Total areal displacement: 1.600000e-02
```

5 Solution Strategy

Kronenfeld et al. (2020) propose the following algorithm: Consider all sequences $A \to B \to C \to D$ of four consecutive vertices in any ring and, for each, find the area-preserving candidate replacement $A \to E \to D$ that minimizes the areal displacement. The candidate with the smallest areal displacement across all rings is then replaced with its three-vertex alternative, provided the replacement does not cause any ring to intersect itself or another ring. After each replacement, the affected candidate sequences are updated and the process is iterated until the target vertex count is reached or no further valid collapse exists. See Kronenfeld et al. (2020) for a detailed derivation of the method and the formula for computing the position of E.

6 Deliverables

1. **Source code.** A C++ implementation in a public GitHub repository. The repository must contain a `Makefile` so that `make` builds the executable `simplify`, and a `README.md` with

instructions for installing any dependencies. The code should be well documented, include comments explaining key functions, and use a commit history that shows systematic progress. Include a set of test cases with known input and expected output to demonstrate correctness. **A set of reference test cases** (input CSV files with matching expected output; parameter settings are in `README.md` within the ZIP archive) **is available on xSITe**; your program's output must match these expected outputs.

2. **Video presentation.** A 10-minute video in which you explain the problem, describe your solution and its key data structures, and present an experimental evaluation of your implementation (including running time and memory usage as a function of input size).

3. **Documentation of AI interaction.** A record of your interactions with generative artificial intelligence (AI) models (e.g., ChatGPT or Claude) during the project, showing the sequence of prompts you used. You may submit this information as a plain text file or a PDF printout. There is no word limit; however, you should include the full text of your prompts and the AI's responses.

4. **AI usage report.** A written, formatted report (not exceeding 2,000 words; state the word count at the end) summarizing your prompting strategies, how the AI's suggestions influenced your design decisions, and where you had to correct or override the AI's output. The report should contain these sections:

   (a) Introduction
   (b) Analysis of AI suggestions
   (c) Reflection on AI assistance
   (d) Distribution of work with AI and within the team

7 Rubric

Correctness of implementation (15 points):
Area preserved to within floating-point tolerance. Topology preserved: no self-intersections, no ring crossings, ring count unchanged. Verified against test datasets provided by the instructors with known expected output. In the GitHub `README.md`, include a section showing the test results.

Areal displacement quality (15 points):
The algorithm produces low areal displacement relative to the number of vertices removed. Evidence of exploring enhancements beyond the basic APSC algorithm, such as non-greedy collapse orderings, look-ahead strategies, or post-processing passes that further reduce areal displacement or Hausdorff distance.

Efficiency (15 points):
Appropriate choice of data structures and algorithms to avoid unnecessary recomputation (e.g., a priority queue for candidate selection and a spatial index for intersection checks). Running time is competitive on large inputs ($\geq$100,000 vertices).

Interactions with generative AI (15 points):
Complete documentation of AI interactions (prompts and responses). Quality and

specificity of questions posed to the AI. Thoughtful analysis of AI-generated suggestions, including what was useful and what was incorrect or misleading. Reflective discussion of how AI assistance shaped the project and where human judgment was essential. Assessment is based on critical evaluation and responsible use of AI suggestions, not on the amount of AI usage.

Code quality and documentation (10 points):
Clean, readable code with meaningful comments. The `make` command builds the project successfully on a fresh checkout. Dependencies clearly documented. Commit history shows systematic progress.

Experimental evaluation (15 points):
Provide your own meaningful test datasets for thorough validation beyond the test cases provided for you on xSITe. For each test dataset, state what property it targets (e.g., a large number of holes, narrow gaps, high vertex count, near-degeneracies) and why it is challenging. Measure running time and peak memory usage. Create plots or tables showing (a) running time vs. input size, (b) peak memory usage vs. input size, and (c) areal displacement vs. target vertex count. Fit functions (e.g., $c \cdot n^k$ or $c \cdot n \log n$) to describe the approximate scaling of running time and memory usage. Discuss and interpret the results.

Video presentation (15 points):
Clear explanation of the problem and solution. Key design decisions and data structures justified. Experimental results presented coherently. Engaging delivery and visually attractive presentation. Within the 10-minute time limit.

References

Kronenfeld, B. J., L. V. Stanislawski, B. P. Buttenfield, and T. Brockmeyer (2020). "Simplification of polylines by segment collapse: minimizing areal displacement while preserving area". In: *International Journal of Cartography* 6.1, pp. 22–46. DOI: 10.1080/23729333.2019.1631535.